

04. 3단계 캐시와 성능 최적화

반복 분석 비용을 줄이되, 저장되지 않은 편집 내용과 파일 변경을 정확하게 반영하도록 캐시 경계를 설계했습니다.

10초

Workspace 목록 TTL

4,000

최대 검색 파일 수

40

분석 결과 캐시 수

800

SourceFile 캐시 수

캐시 계층 구조

각 계층은 서로 다른 비용을 줄이며, 상위 캐시가 실패하면 하위 계층의 재사용 여부를 판단합니다.

분석 요청

Active Editor + Graph Options

Layer 1

Workspace File List Cache

- ts / tsx / js / jsx 파일 목록을 10초 동안 재사용
- node_modules, dist, build, out, .next 제외 · 최대 4,000개 검색

Cache Key

workspace state

Layer 2

Analysis Result Cache

- 동일한 코드와 분석 옵션에서 graph · diagnostics · trace 결과 재사용
- 최대 40개 결과를 최근 접근 시각 기준으로 유지

codeHash

graphDepth

traceMode

workspaceFilesHash

분석 결과

graph · diagnostics
trace · meta

Layer 3

TypeScript SourceFile Cache

- 변경되지 않은 디스크 파일의 파싱 결과를 CompilerHost에서 재사용
- 최대 800개 SourceFile을 최근 접근 시각 기준으로 관리

mtimeMs

size

ScriptKind

languageVersionKey

정확성을 보장하는 예외 처리

Active Editor Buffer

저장되지 않은 편집 내용은
항상 현재 메모리 텍스트로 분석

Disk SourceFile Cache

활성 파일은 디스크 캐시를 우회하고
주변 파일만 캐시에서 재사용

Stale Data 방지

현재 편집 내용은 최신 상태를 유지하면서
변경되지 않은 의존 파일의 파싱 비용만 절감

캐시 무효화 조건

파일 시스템과 설정 변경을 감지해 분석 결과 캐시와 SourceFile 캐시를 명시적으로 비웁니다.

문서 내용 변경

onDidChangeTextDocument

문서 저장

onDidSaveTextDocument

파일 생성 · 삭제 · 이름 변경

FileSystemWatcher

tsconfig 변경

create · change · delete

캐시 설정 변경

analysisCache.enabled

캐시 적중률만 높이는 것이 아니라, 변경 감지와 메모리 버퍼 우회를 함께 설계해 성능과 정확성을 동시에 확보했습니다.